

Homework 4

Due: 11:59pm on April 15, 2025 (Tuesday)

Note: This assignment may take more time than it would appear at first glance. Start early!

Objectives:

- Have a deeper understanding about how HashTable works by implementing it from scratch.
- Get a chance to think about what makes a good hash function.

In Homeworks 1 and 3, you implemented MyArray and SortedLinkedList to store words parsed from a text file. This time, your goal is to make search very fast. So, your plan is to implement a new data structure class:

```
public class MyHashTable implements MyHTInterface
```

MyHashTable must have the following properties and capabilities.

- This is not an exercise for recursion. **Do not use recursion.**
- It implements MyHTInterface that is provided to you. Examine this interface carefully.
- It uses an array as its underlying data structure and each element of the array is DataItem (private static nested class). Each DataItem has String as its data value and int value to keep track of the frequency of its data value. That means, if you add the same data value (string) twice, the frequency of the data becomes 2. You must use the following code for the underlying array in your code.

```
private DataItem[] hashArray;
```

- Do not use any nested class other than the DataItem class.
- As you learned during lecture, this class does not care about the order of elements because our focus is to make search fast.
- It uses **linear probing** as its collision resolution mechanism.
- It has the default capacity of 10 and the default load factor of 0.5. Along with a no-arg constructor, it should also have another constructor that allows the clients to pass an initial capacity value. If the initial capacity is less than or equal to 0, it should complain by throwing a runtime exception. To make it simpler, we won't allow the user to pass a custom load factor (and instead rely on the default load factor).
- Its hash function should be implemented based on the algorithm shown in Module 11. For example, the hash value of "cats" is: $(3 \cdot 27^3 + 1 \cdot 27^2 + 20 \cdot 27^1 + 19 \cdot 27^0) = 60,337 \% \text{tableLength}$.
- However, it recognizes the importance of making the hash function more efficient. For that purpose, **Horner's method** (http://en.wikipedia.org/wiki/Horner_scheme) must be used in its hash function as follows.

$$\text{var4} \cdot n^4 + \text{var3} \cdot n^3 + \text{var2} \cdot n^2 + \text{var1} \cdot n^1 + \text{var0} \cdot n^0$$

can be rewritten as follows using Horner's method

$$(((\text{var4} \cdot n + \text{var3}) \cdot n + \text{var2}) \cdot n + \text{var1}) \cdot n + \text{var0}$$

- It should rehash the current hash table so that the load factor stays within (less than **or equal to**) 0.5 at any point. The load factor is the ratio of the number of data items (not including deleted flags) in a hash table to the length of the array.

- As it rehashes, it should print how many items are now being rehashed and what the new length of the table is to users. (Check the expected results while testing your program with `HTTest.java` file.)
- Also, when rehashing, increase the underlying array by twice bigger than the current length. However, if the new length is not a prime number, then you should use the next prime number that is larger than the new length. For example, if your current array length is 2 and you need to rehash, 4 is twice bigger than the current array length but you need to use 5 instead because 4 is not a prime number and 5 is the next prime number.
- While rehashing, think about how you would deal with those deleted items. Would it make sense to keep them or not?
- Unlike Map or Set interface of the Java Collections Framework, it has a few useful (?) methods that can be helpful for a better understanding of the hash table, such as getting the number of collisions and getting the hash value of any string.
- To not waste any time, the calculation of size (number of items) and number of collisions should occur while inserting a new element or rehashing. As a result, when the user calls the `size()` method or the `numOfCollisions()` method, the running time complexity is $O(1)$.
- If you delete an existing string with frequency 2, the final frequency of the string becomes 0. That means, no matter what the frequency of the word is, the delete operation should mark the space as deleted (instead of decreasing the frequency by one). And, of course, the size (number of items) should be updated accordingly.
 - But, to make things simpler, you do not need to worry about updating (more specifically, reducing) the number of collisions when remove method is invoked.

Just like Homeworks 1 and 3, **you are NOT allowed to use any of the Collections Framework, which means there should be NO imports in your source code.** To make things simpler, you can assume that your clients know that they need to convert all the words in a file to lowercase before inserting them into your hash table. Thus, you should not convert strings to lowercases in your implementation.

As in previous assignments, a word is a sequence of letters `[a..zA..Z]`, does not include digits `[0..9]`, and does not include other characters than the sequence of letters `[a..zA..Z]`. Even though the word definition here talks about uppercase letters (this is to make the definition be consistent with other assignments), **you can assume that we won't test your code with any uppercase letters.** Also, your implementation should not make any assumption about the length of words. Some examples of non-words include `abc123`, `a12bc`, `1234`, `ab_c`, `abc_`.

In the case of non-words, do not throw an exception. Instead, gently ignore any non-words in your implementation.

Testing:

There will be one test case provided to you, `HTTest.java`. **Your output should match the output in the expected result comment section in the `HTTest.java` file.** There are a lot more cases that you should think about and test on your own. You should also test your program on your own with other text files.

Be careful!

- When the `display()` method is called, `DataItem` should show its String value and its frequency as pair, “[Value, Frequency]”, and the empty space is represented with “***” and any space that used to have an item but deleted later should be represented as “#DEL#”. Please refer to expected results of `HTTest.java` file to make sure your implementation is on the right track.

- We recommend spending some time to think about how to count the number of collisions. The definition of collision you are to rely on is: “Two different keys map to **the same hash value**.” Checking the code for Lab 4 could be helpful.

There is MyHTInterface interface where you can find about the methods to be implemented and their descriptions. Read its JavaDoc comments very carefully!

Deliverables:

- Submit your MyHashTable.java file using Autolab (<https://autolab.andrew.cmu.edu>). Do not zip it, and do not use package statements. The test code provided to you is a starting point. Test extensively on your own!

Grading:

Autolab will grade your assignment as follows.

- Working code: 90 points
- Coding conventions: 10 points
 - We will deduct one point for each coding convention issue detected.

Autolab will show you the results of its grading within approximately a few minutes of your submission. You may submit multiple times to correct any problems with your assignment. Autolab uses the last submission as your grade.

The Autolab score is not final. The TAs will look at your source code for correctness and design, and deduct points if applicable. The most important criterion is always correctness. It is also critical that your code is efficient and follows the specification properly. Additionally, it should be readable and well organized. This includes proper use of clear comments. Points will be deducted for poor design decisions and unreadable code, etc. Common sources of deductions from past semesters are shown below. Note that this list is not exhaustive and is merely intended to draw your attention to parts easy to overlook with this assignment; you may lose points if the TAs find substantial issues with your code outside of this list.

10 pts	• Not using Horner’s method for the hash function • Starting search at index 0 instead of the initial hash value for contains() or insert() • Performing String concatenations in a loop (instead of using a StringBuilder properly) • Not properly overwriting a deleted index space; think about what should go in a deleted slot ◦ Make sure you declare this item as a constant
3-5 pts	• Implementing DataItem without proper nesting or access modifiers (etc.) • Implementing inefficient size() or numOfCollisions() methods; should be O(1) • Creating a new HashTable instance while rehashing • Not declaring the default capacity and load factor as constants in HashTable • Calling contains() in other methods (e.g., insert()); do you need to do this extra work? • Having non-private helper methods • Not having meaningful Javadoc comments • Having unaddressed warnings in code • Not removing dead code

As mentioned in the syllabus, **late submissions will NOT be accepted**. If you have multiple versions of your code file, make sure you submit the correct version.